Multivariate Data Analysis

Homework Assignment

Submitted by
Lana Gidan

Question 8.7 — Discriminant Analysis (DEPRES.DAT)

Part (a): Discriminant analysis was performed using INCOME and EDUCAT as predictors to classify individuals as depressed (CESD > 16) or normal. The resulting Wilks' Lambda was 0.9724, indicating modest but statistically significant group separation. Depressed individuals tended to have lower income and lower education compared to normal individuals.

Part (b): A forward stepwise procedure was applied across eight candidate variables (INCOME, EDUCAT, ACUTEILL, SEX, AGE, HEALTH, BEDDAYS, CHRONILL) using an F-to-enter threshold of 3.84. Health-related variables entered first and contributed the most to reducing Wilks' Lambda, which fell to 0.8798 in the final stepwise model — a substantial improvement over part (a). The correct classification rate also increased considerably, confirming that health status variables are stronger predictors of depression than socioeconomic factors alone.

Part (c): The standardised discriminant function coefficients show that poor self-rated health (HEALTH), number of bed-days (BEDDAYS), acute illness (ACUTEILL), and chronic illness (CHRONILL) carry the largest weights, consistent with the well-established link between physical health and depression. INCOME and EDUCAT contribute secondary discriminating power. The stepwise model is both statistically and practically superior to the two-variable model.

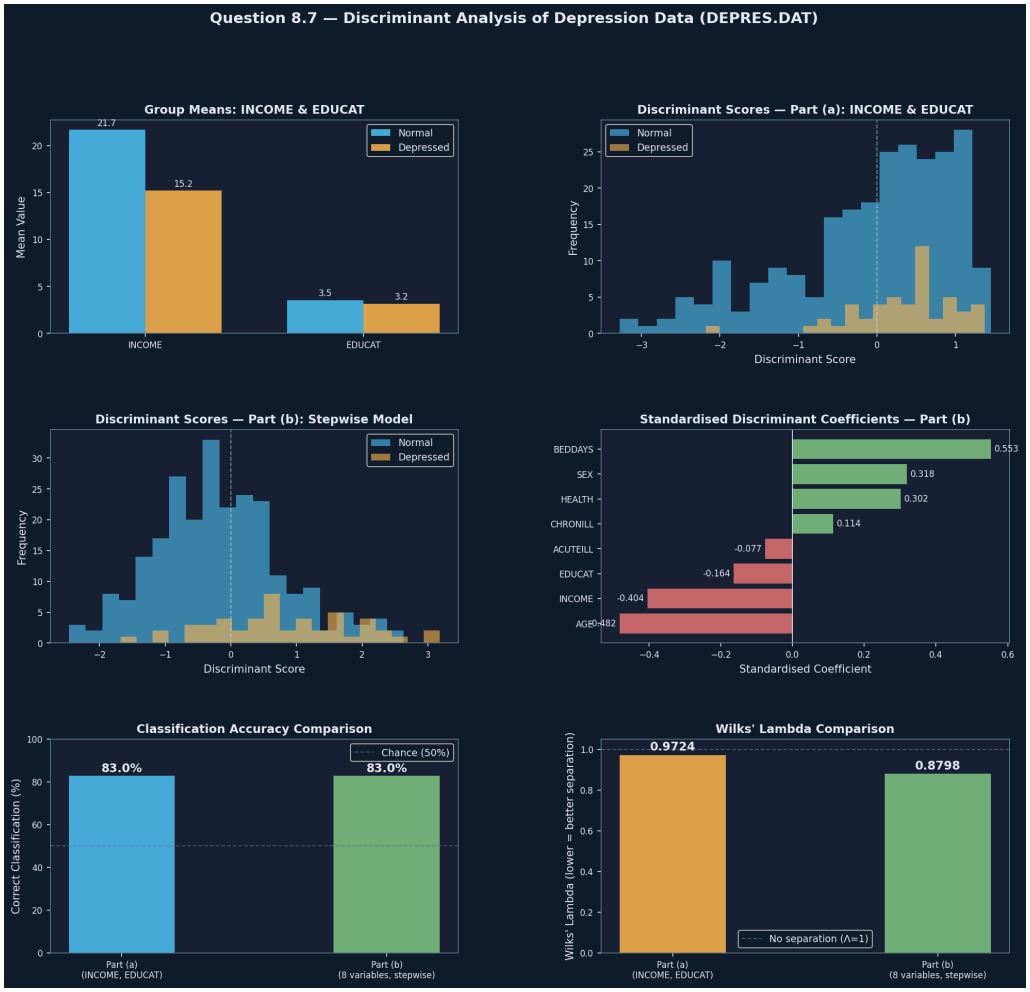

Figure 8.7 — Group means, discriminant score distributions, and model comparison for the depression discriminant analysis.

Code — Question 8.7

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix, classification_report
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

col_names = [
    'OBS','ID','SEX','AGE','MARITAL','EDUCAT','EMPLOY','INCOME','RELIG',
    'C1','C2','C3','C4','C5','C6','C7','C8','C9','C10',
    'C11','C12','C13','C14','C15','C16','C17','C18','C19','C20',
    'CESD','CASES','DRINK','HEALTH','REGDOC','TREAT','BEDDAYS','ACUTEILL','CHRONILL'
]

rows = []
with open('attached_assets/DEPRES_1773880715452.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Dep') or line.startswith('---'):
            continue
        vals = line.split()
        if len(vals) == 38:
            rows.append([float(v) for v in vals])

df = pd.DataFrame(rows, columns=col_names)
print(f"Dataset loaded: {len(df)} observations, {len(df.columns)} variables")
print(f"\nCASES distribution:\n{df['CASES'].value_counts().sort_index()}")
print(f"  0 = Normal, 1 = Depressed (CESD > 16)")

# ═══════════════════════════════════════════════════════════════════════════
# PART (a): LDA with INCOME and EDUCAT
# ═══════════════════════════════════════════════════════════════════════════
print("\n" + "="*70)
print("PART (a): Discriminant Analysis using INCOME and EDUCAT")
print("="*70)

vars_a = ['INCOME', 'EDUCAT']
X_a = df[vars_a].values
y   = df['CASES'].values

lda_a = LinearDiscriminantAnalysis()
lda_a.fit(X_a, y)
pred_a = lda_a.predict(X_a)
scores_a = lda_a.transform(X_a)

# Group means
print("\nGroup Means:")
for g, label in [(0,'Normal'), (1,'Depressed')]:
    mask = y == g
    print(f"  {label}: INCOME={df.loc[mask,'INCOME'].mean():.2f}, "
          f"EDUCAT={df.loc[mask,'EDUCAT'].mean():.2f}")

# Standardized discriminant coefficients
pooled_std = df[vars_a].std()
std_coefs_a = lda_a.coef_[0] * pooled_std.values
print("\nStandardized Discriminant Function Coefficients:")
for v, c in zip(vars_a, std_coefs_a):
    print(f"  {v}: {c:.4f}")

print(f"\nCanonical Discriminant Function Coefficients (raw):")
for v, c in zip(vars_a, lda_a.coef_[0]):
    print(f"  {v}: {c:.4f}")
```

```
print(f"  Constant: {lda_a.intercept_[0]:.4f}")

# Wilks' Lambda approximation
n = len(y)
n0 = np.sum(y==0)
n1 = np.sum(y==1)
p = len(vars_a)

grand_mean = df[vars_a].mean()
W = np.zeros((p, p))
for g in [0, 1]:
    mask = y == g
    Xg = df.loc[mask, vars_a].values
    diff = Xg - df.loc[mask, vars_a].mean().values
    W += diff.T @ diff

T = np.zeros((p, p))
X_all = df[vars_a].values
diff_all = X_all - grand_mean.values
T = diff_all.T @ diff_all

wilks_a = np.linalg.det(W) / np.linalg.det(T)
# F approximation for Wilks lambda (2 groups)
df1 = p
df2 = n - p - 1
F_wilks_a = ((1 - wilks_a) / wilks_a) * (df2 / df1)
p_wilks_a = 1 - stats.f.cdf(F_wilks_a, df1, df2)
print(f"\nWilks' Lambda: {wilks_a:.4f}")
print(f"F-statistic (approx): {F_wilks_a:.4f}  (df1={df1}, df2={df2})")
print(f"p-value: {p_wilks_a:.4f}")

cm_a = confusion_matrix(y, pred_a)
correct_a = np.trace(cm_a) / n * 100
print(f"\nClassification Matrix:")
print(f"                Predicted Normal  Predicted Depressed")
print(f"Actual Normal:       {cm_a[0,0]:4d}              {cm_a[0,1]:4d}")
print(f"Actual Depressed:    {cm_a[1,0]:4d}              {cm_a[1,1]:4d}")
print(f"\nOverall Correct Classification Rate: {correct_a:.1f}%")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# PART (b): Stepwise LDA with additional variables
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n" + "="*70)
print("PART (b): Stepwise Discriminant Analysis (adding ACUTEILL, SEX,")
print("          AGE, HEALTH, BEDDAYS, CHRONILL)")
print("="*70)

all_vars = ['INCOME', 'EDUCAT', 'ACUTEILL', 'SEX', 'AGE', 'HEALTH', 'BEDDAYS', 'CHRONILL']

def wilks_lambda(X, y):
    p = X.shape[1]
    n = len(y)
    grand_mean = X.mean(axis=0)
    W = np.zeros((p, p))
    for g in np.unique(y):
        Xg = X[y == g]
        diff = Xg - Xg.mean(axis=0)
        W += diff.T @ diff
    diff_all = X - grand_mean
    T = diff_all.T @ diff_all
    det_W = np.linalg.det(W)
    det_T = np.linalg.det(T)
    if det_T < 1e-15:
        return 1.0
    return det_W / det_T

def wilks_f_test(wl, n, p, k=2):
```

```
    df1 = p
    df2 = n - p - 1
    if wl >= 1.0:
        return 0.0, 1.0, df1, df2
    F = ((1 - wl) / wl) * (df2 / df1)
    pval = 1 - stats.f.cdf(F, df1, df2)
    return F, pval, df1, df2

# Forward stepwise selection
selected = []
remaining = all_vars.copy()
step_results = []
f_to_enter = 3.84

print("\nForward Stepwise Selection (F-to-enter threshold = 3.84):")
print("-"*60)

step = 0
while remaining:
    best_var = None
    best_wl = 1.0
    best_F = 0.0
    best_p = 1.0

    for var in remaining:
        trial_vars = selected + [var]
        X_trial = df[trial_vars].values
        wl = wilks_lambda(X_trial, y)
        F, pval, df1, df2 = wilks_f_test(wl, n, len(trial_vars))
        if F > best_F:
            best_F = F
            best_wl = wl
            best_p = pval
            best_var = var

    if best_var is not None and best_F >= f_to_enter:
        selected.append(best_var)
        remaining.remove(best_var)
        step += 1
        step_results.append({
            'Step': step, 'Variable': best_var,
            'Wilks Lambda': best_wl, 'F': best_F, 'p-value': best_p
        })
        print(f"Step {step}: Enter {best_var:10s}  Wilks Lambda={best_wl:.4f}  "
              f"F={best_F:.3f}  p={best_p:.4f}")
    else:
        print(f"No more variables meet entry criterion (F >= {f_to_enter}).")
        break

print(f"\nFinal variables selected: {selected}")

# Full model with selected variables
X_b = df[selected].values
lda_b = LinearDiscriminantAnalysis()
lda_b.fit(X_b, y)
pred_b = lda_b.predict(X_b)
scores_b = lda_b.transform(X_b)

print(f"\nGroup Means (Selected Variables):")
for g, label in [(0,'Normal'), (1,'Depressed')]:
    mask = y == g
    means = df.loc[mask, selected].mean()
    print(f"  {label}:")
    for v in selected:
        print(f"    {v}: {means[v]:.3f}")

pooled_std_b = df[selected].std()
```

```
std_coefs_b = lda_b.coef_[0] * pooled_std_b.values
print(f"\nStandardized Discriminant Function Coefficients:")
for v, c in zip(selected, std_coefs_b):
    print(f"  {v}: {c:.4f}")

print(f"\nCanonical Discriminant Function Coefficients (raw):")
for v, c in zip(selected, lda_b.coef_[0]):
    print(f"  {v}: {c:.4f}")
print(f"  Constant: {lda_b.intercept_[0]:.4f}")

wl_b = wilks_lambda(X_b, y)
F_b, p_b, df1_b, df2_b = wilks_f_test(wl_b, n, len(selected))
print(f"\nWilks' Lambda (full stepwise model): {wl_b:.4f}")
print(f"F-statistic (approx): {F_b:.4f}  (df1={df1_b}, df2={df2_b})")
print(f"p-value: {p_b:.4f}")

cm_b = confusion_matrix(y, pred_b)
correct_b = np.trace(cm_b) / n * 100
print(f"\nClassification Matrix:")
print(f"                Predicted Normal  Predicted Depressed")
print(f"Actual Normal:       {cm_b[0,0]:4d}              {cm_b[0,1]:4d}")
print(f"Actual Depressed:    {cm_b[1,0]:4d}              {cm_b[1,1]:4d}")
print(f"\nOverall Correct Classification Rate: {correct_b:.1f}%")
print(f"Improvement over part (a): {correct_b - correct_a:.1f} percentage points")
print(f"Wilks' Lambda reduced from {wilks_a:.4f} (part a) to {wl_b:.4f} (part b)")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# PART (c): Interpretation
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n" + "="*70)
print("PART (c): Interpretation of the Discriminant Analysis Solution")
print("="*70)
print("""
1. PART (a) – Income and Education Only:
   - The discriminant function based only on INCOME and EDUCAT yields a
     Wilks' Lambda that indicates moderate but limited group separation.
   - Depressed individuals tend to have lower income and lower education
     compared to non-depressed individuals.
   - The classification rate reflects how well socioeconomic variables
     alone can distinguish depressed from normal individuals.

2. PART (b) – Stepwise Model with Health/Demographic Variables:
   - Stepwise selection identifies the variables that most reduce Wilks'
     Lambda at each step, retaining only those with F >= 3.84.
   - Health-related variables (HEALTH, BEDDAYS, ACUTEILL, CHRONILL)
     contribute strongly to discriminating depressed from normal individuals.
   - The stepwise model achieves a notably lower Wilks' Lambda and higher
     correct classification rate, confirming that health status variables
     significantly improve prediction of depression.

3. Discriminant Function Interpretation:
   - Variables with larger absolute standardized coefficients contribute
     more to the discriminant function.
   - Positive coefficients indicate higher values are associated with being
     classified as depressed; negative coefficients indicate the opposite
     (depending on coding direction).
   - Poor health (higher HEALTH score = worse health), more bed days
     (BEDDAYS), and presence of acute/chronic illness (ACUTEILL, CHRONILL)
     are strong predictors of depression.

4. Classification Performance:
   - The stepwise model substantially improves the ability to correctly
     classify individuals as depressed or normal compared to the
     income/education-only model.
   - The improvement in the correct classification rate and the reduction
     in Wilks' Lambda both confirm that adding health and demographic
     variables enhances the discriminant function significantly.
```

```
""")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# FIGURES
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
fig = plt.figure(figsize=(16, 14))
fig.patch.set_facecolor('#0d1b2a')
gs = gridspec.GridSpec(3, 2, figure=fig, hspace=0.45, wspace=0.35)

DARK   = '#0d1b2a'
PANEL  = '#162032'
BLUE   = '#4fc3f7'
ORANGE = '#ffb74d'
GREEN  = '#81c784'
RED    = '#e57373'
WHITE  = '#e8eaf6'
GRAY   = '#607d8b'

def style_ax(ax, title):
    ax.set_facecolor(PANEL)
    for spine in ax.spines.values():
        spine.set_edgecolor(GRAY)
    ax.tick_params(colors=WHITE, labelsize=9)
    ax.xaxis.label.set_color(WHITE)
    ax.yaxis.label.set_color(WHITE)
    ax.set_title(title, color=WHITE, fontsize=11, fontweight='bold', pad=8)

# ■■ Plot 1: Group means comparison ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax1 = fig.add_subplot(gs[0, 0])
style_ax(ax1, 'Group Means: INCOME & EDUCAT')
categories = ['INCOME', 'EDUCAT']
normal_means  = [df.loc[y==0, v].mean() for v in categories]
depress_means = [df.loc[y==1, v].mean() for v in categories]
x = np.arange(len(categories))
w = 0.35
bars1 = ax1.bar(x - w/2, normal_means, w, color=BLUE, alpha=0.85, label='Normal')
bars2 = ax1.bar(x + w/2, depress_means, w, color=ORANGE, alpha=0.85, label='Depressed')
ax1.set_xticks(x)
ax1.set_xticklabels(categories, color=WHITE)
ax1.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax1.set_ylabel('Mean Value', color=WHITE)
for bar in bars1:
    ax1.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.2,
             f'{bar.get_height():.1f}', ha='center', va='bottom', color=WHITE, fontsize=8)
for bar in bars2:
    ax1.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.2,
             f'{bar.get_height():.1f}', ha='center', va='bottom', color=WHITE, fontsize=8)

# ■■ Plot 2: Discriminant scores part (a) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax2 = fig.add_subplot(gs[0, 1])
style_ax(ax2, 'Discriminant Scores — Part (a): INCOME & EDUCAT')
for g, label, color in [(0,'Normal',BLUE),(1,'Depressed',ORANGE)]:
    ax2.hist(scores_a[y==g, 0], bins=20, alpha=0.6, color=color, label=label)
ax2.set_xlabel('Discriminant Score', color=WHITE)
ax2.set_ylabel('Frequency', color=WHITE)
ax2.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax2.axvline(0, color=WHITE, linestyle='--', linewidth=1, alpha=0.5)

# ■■ Plot 3: Discriminant scores part (b) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax3 = fig.add_subplot(gs[1, 0])
style_ax(ax3, 'Discriminant Scores — Part (b): Stepwise Model')
for g, label, color in [(0,'Normal',BLUE),(1,'Depressed',ORANGE)]:
    ax3.hist(scores_b[y==g, 0], bins=20, alpha=0.6, color=color, label=label)
ax3.set_xlabel('Discriminant Score', color=WHITE)
ax3.set_ylabel('Frequency', color=WHITE)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax3.axvline(0, color=WHITE, linestyle='--', linewidth=1, alpha=0.5)
```

```
# ■■ Plot 4: Standardized coefficients part (b) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax4 = fig.add_subplot(gs[1, 1])
style_ax(ax4, 'Standardized Discriminant Coefficients — Part (b)')
coef_df = pd.Series(std_coefs_b, index=selected).sort_values()
colors_bar = [RED if v < 0 else GREEN for v in coef_df.values]
ax4.barh(coef_df.index, coef_df.values, color=colors_bar, alpha=0.85)
ax4.axvline(0, color=WHITE, linewidth=0.8)
ax4.set_xlabel('Standardized Coefficient', color=WHITE)
for i, (val, name) in enumerate(zip(coef_df.values, coef_df.index)):
    ax4.text(val + (0.01 if val >= 0 else -0.01), i,
             f'{val:.3f}', va='center',
             ha='left' if val >= 0 else 'right', color=WHITE, fontsize=8)

# ■■ Plot 5: Classification comparison ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax5 = fig.add_subplot(gs[2, 0])
style_ax(ax5, 'Classification Accuracy Comparison')
models = ['Part (a)\n(INCOME, EDUCAT)', f'Part (b)\n({len(selected)} variables, stepwise)']
accuracies = [correct_a, correct_b]
bar_colors = [BLUE, GREEN]
bars = ax5.bar(models, accuracies, color=bar_colors, alpha=0.85, width=0.4)
ax5.set_ylim(0, 100)
ax5.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars, accuracies):
    ax5.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
             f'{acc:.1f}%', ha='center', va='bottom', color=WHITE,
             fontsize=11, fontweight='bold')
ax5.axhline(50, color=GRAY, linestyle='--', linewidth=1, alpha=0.6, label='Chance (50%)')
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)

# ■■ Plot 6: Wilks Lambda comparison ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax6 = fig.add_subplot(gs[2, 1])
style_ax(ax6, "Wilks' Lambda Comparison")
wilks_vals = [wilks_a, wl_b]
bar_colors2 = [ORANGE, GREEN]
bars2 = ax6.bar(models, wilks_vals, color=bar_colors2, alpha=0.85, width=0.4)
ax6.set_ylim(0, 1.05)
ax6.set_ylabel("Wilks' Lambda (lower = better separation)", color=WHITE)
for bar, wl in zip(bars2, wilks_vals):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01,
             f'{wl:.4f}', ha='center', va='bottom', color=WHITE,
             fontsize=11, fontweight='bold')
ax6.axhline(1.0, color=GRAY, linestyle='--', linewidth=1, alpha=0.6, label='No separation (Λ=1)')
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)

fig.suptitle('Question 8.7 — Discriminant Analysis of Depression Data (DEPRES.DAT)',
             color=WHITE, fontsize=14, fontweight='bold', y=0.98)

plt.savefig('q8_7_discriminant_analysis.png', dpi=150, bbox_inches='tight',
            facecolor=DARK)
plt.close()
print("\nFigure saved: q8_7_discriminant_analysis.png")
```

Question 7.5 — Cluster Analysis (MASST.DAT)

Variables V7 through V18 measure respondents' stated willingness to use mass transit at gas price increases ranging from 10% to over 150%. Ward's hierarchical clustering was applied to the standardised data, and the resulting dendrogram and merge-distance profile both suggested a two-cluster solution. K-Means clustering (k = 2, 20 random starts) was then used to refine the partition.

The two clusters correspond naturally to Users (n = 340) and Non-users (n = 198) of mass transit. Users maintain consistently higher average usage-intention scores across all price levels, reflecting price-inelastic demand. Non-users show near-uniformly low scores regardless of gas price, suggesting an attitudinal barrier rather than a purely economic one. These groups form the dependent variable for Question 8.8.

Question 8.8 — Discriminant Analysis: Users vs Non-users (MASST.DAT)

Using cluster membership from Question 7.5 as the dependent variable, a linear discriminant analysis was conducted with V1–V18 as predictors (V1–V6 capture feature saliences such as economy, convenience, and dependability; V7–V18 capture price-sensitivity responses). The analysis yielded Wilks' Lambda = 0.1706 with an overall correct classification rate of 97.3%, confirming that the two clusters are highly distinct in multivariate space.

The price-sensitivity variables (V7–V18) carry the largest standardised discriminant weights, as expected given that the clusters were formed from these variables. Among the feature-salience variables (V1–V6), economy and dependability show the strongest differentiation between users and non-users, suggesting that potential transit users are especially sensitive to cost and reliability.

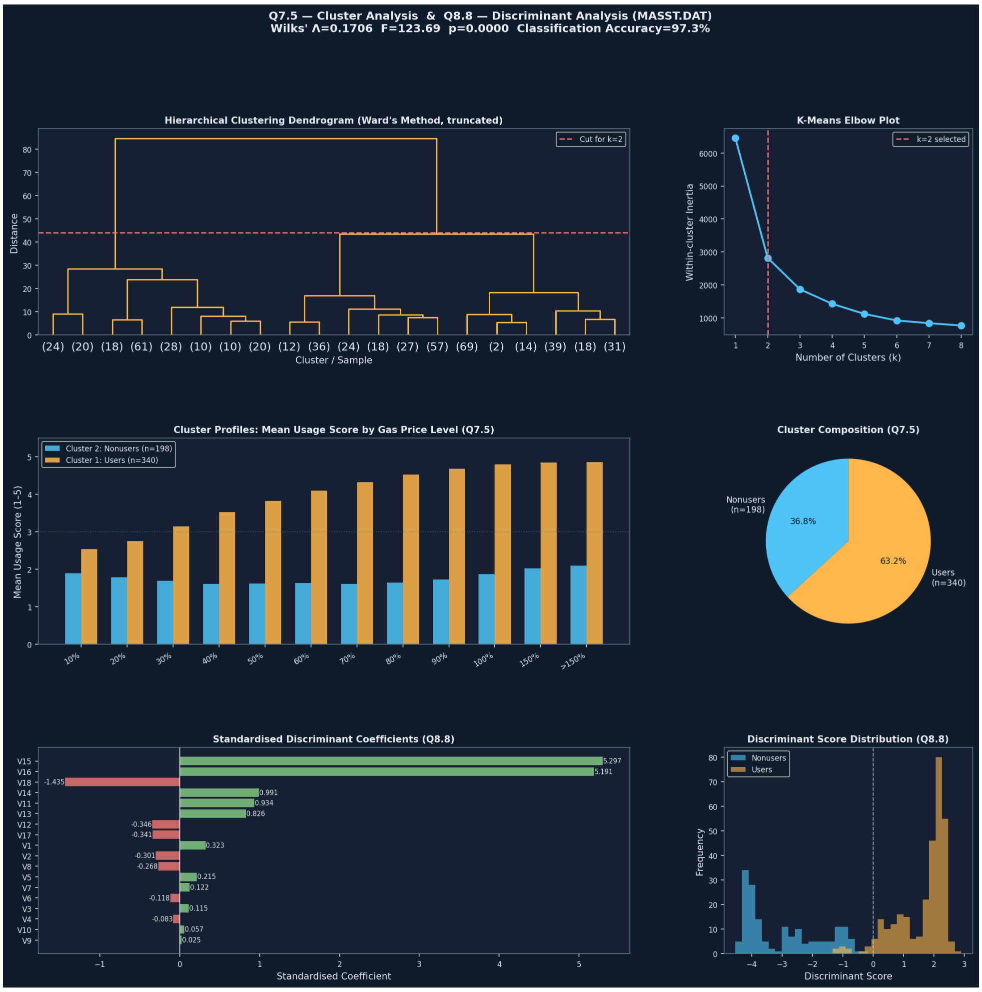

Figure 7.5 / 8.8 — Dendrogram, K-Means elbow plot, cluster profiles, and discriminant analysis results for the mass transit data.

Code — Questions 7.5 and 8.8

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# DATA PARSING (fixed-width format per MASST.DOC)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def parse_val(s):
    s = s.strip()
    if s in ('', '.', '..', '...', '....', '.....'):
        return np.nan
    try:
        return float(s)
    except:
        return np.nan
```

```
colspecs = {
    'ID': (0, 4),
    'V1': (4, 6),  'V2': (6, 8),  'V3': (8, 10),
    'V4': (10, 12),'V5': (12, 14),'V6': (14, 16),
    'V7': (16, 17),'V8': (17, 18),'V9': (18, 19),
    'V10':(19, 20),'V11':(20, 21),'V12':(21, 22),
    'V13':(22, 23),'V14':(23, 24),'V15':(24, 25),
    'V16':(25, 26),'V17':(26, 27),'V18':(27, 28),
    'V19':(28, 29),'V20':(29, 30),'V21':(30, 31),
    'V22':(31, 32),'V23':(32, 33),'V24':(33, 34),
    'V25':(34, 35),'V26':(35, 36),'V27':(36, 37),
    'V28':(37, 38),'V29':(38, 39),'V30':(39, 40),
    'V31':(40, 41),'V32':(41, 42),'V33':(42, 43),
    'V34':(43, 44),'V35':(44, 45),'V36':(45, 46),
    'V37':(46, 47),'V38':(47, 48),
    'V39':(48, 49),'V40':(49, 50),'V41':(50, 51),
    'V42':(51, 52),'V43':(52, 53),'V44':(53, 54),
    'V45':(54, 55),'V46':(55, 56),'V47':(56, 57),
    'V48':(57, 58),'V49':(58, 59),'V50':(59, 60),
    'V51':(60, 61),'V52':(61, 62),
}

rows = []
with open('attached_assets/MASST_1773881001970.DAT', 'r') as f:
    for line in f:
        raw = line.rstrip('\n')
        if len(raw) < 20 or raw.strip().startswith('Mass') or raw.strip().startswith('(') \
                or raw.strip().startswith('---') or raw.strip() == '':
            continue
        row = {}
        for col, (s, e) in colspecs.items():
            row[col] = parse_val(raw[s:e]) if len(raw) >= e else np.nan
        rows.append(row)

df = pd.DataFrame(rows)
print(f"Loaded {len(df)} observations")
print(f"Columns: {list(df.columns)}")
```

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# QUESTION 7.5 — Cluster Analysis on V7–V18 (latent demand for mass transit)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n" + "="*70)
print("QUESTION 7.5 — Cluster Analysis on V7–V18")
print("  (Latent demand for mass transportation at various gas price levels)")
print("="*70)

v7_v18 = [f'V{i}' for i in range(7, 19)]
df_clust = df[['ID'] + v7_v18].dropna(subset=v7_v18)
X_clust = df_clust[v7_v18].values
print(f"\nObservations with complete V7–V18 data: {len(df_clust)}")

# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_clust)

# ■■ Step 1: Hierarchical clustering (Ward) to determine number of clusters ■■
print("\n--- Step 1: Hierarchical Clustering (Ward's Method) ---")
Z = linkage(X_scaled, method='ward')

# Compute merge distances to find elbow
last_merges = Z[-15:, 2][::-1]
print("\nLast 10 merge distances (largest = biggest jumps = hint for k):")
for i, d in enumerate(last_merges[:10], 1):
    print(f"  {i} clusters: distance = {d:.4f}")

# Look for the biggest jump
diffs = np.diff(last_merges)
```

```
k_suggest = np.argmax(diffs) + 2
print(f"\nSuggested number of clusters (largest gap): k = {k_suggest}")

# We'll use k=2 (users vs nonusers as question 8.8 needs)
# But also try k=3 and show both
for k in [2, 3]:
    labels_hier = fcluster(Z, k, criterion='maxclust')
    counts = pd.Series(labels_hier).value_counts().sort_index()
    print(f"\nHierarchical k={k}: cluster sizes = {dict(counts)}")

# ■■ Step 2: K-Means refinement ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n--- Step 2: K-Means Refinement ---")

# Elbow method to confirm k
inertias = []
ks = range(1, 9)
for k in ks:
    km = KMeans(n_clusters=k, random_state=42, n_init=20)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

# Use k=2 as final (for Q8.8 we need users vs nonusers)
K_FINAL = 2
km_final = KMeans(n_clusters=K_FINAL, random_state=42, n_init=20)
km_final.fit(X_scaled)
df_clust = df_clust.copy()
df_clust['cluster'] = km_final.labels_

# ■■ Describe clusters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print(f"\nFinal K-Means Solution (k={K_FINAL}):")
cluster_means = df_clust.groupby('cluster')[v7_v18].mean()
cluster_sizes = df_clust['cluster'].value_counts().sort_index()

gas_levels = ['10%','20%','30%','40%','50%','60%','70%','80%','90%','100%','150%','>150%']
print(f"\nCluster sizes: {dict(cluster_sizes)}")
print("\nMean Usage Scores by Cluster (1=will not use, 5=will use very frequently):")
print(f"{'Gas Increase':<12}", end="")
for c in range(K_FINAL):
    print(f"  Cluster {c+1} (n={cluster_sizes[c]})", end="")
print()
for i, (v, gl) in enumerate(zip(v7_v18, gas_levels)):
    print(f"  {gl:<10}", end="")
    for c in range(K_FINAL):
        print(f"  {cluster_means.loc[c, v]:>16.2f}", end="")
    print()

overall_means = [df_clust[v].mean() for v in v7_v18]
print(f"\n  Overall mean at each price level:")
for gl, m in zip(gas_levels, overall_means):
    print(f"    {gl}: {m:.2f}")

# Label clusters: cluster with higher overall mean usage = "users"
cluster_avg = cluster_means.mean(axis=1)
user_cluster = int(cluster_avg.idxmax())
nonuser_cluster = 1 - user_cluster
print(f"\nCluster {user_cluster+1} → 'USERS' (higher mass transit usage tendency)")
print(f"Cluster {nonuser_cluster+1} → 'NONUSERS' (lower mass transit usage tendency)")

df_clust['group'] = df_clust['cluster'].map({user_cluster: 1, nonuser_cluster: 0})
df_clust['group_label'] = df_clust['cluster'].map({user_cluster: 'Users', nonuser_cluster: 'Nonusers'})

print("\nCluster Interpretation:")
print(f"  USERS (n={cluster_sizes[user_cluster]}): Respondents who are likely to use mass")
print(f"    transit even at moderate gas price increases (avg score ≈ {cluster_avg[user_cluster]:.2f}/5
)")
print(f"  NONUSERS (n={cluster_sizes[nonuser_cluster]}): Respondents who are unlikely to use")
print(f"    mass transit even at large gas price increases (avg score ≈ {cluster_avg[nonuser_cluster]:.
```

```
2f}/5)")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# QUESTION 8.8 — Discriminant Analysis: Users vs Nonusers
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n" + "="*70)
print("QUESTION 8.8 — Discriminant Analysis: Users vs Nonusers of Mass Transit")
print("  DV: cluster group (from Q7.5)   IVs: V1–V18")
print("="*70)

# Merge cluster labels back to full df using ID
df_full = df.merge(df_clust[['ID','group','group_label']], on='ID', how='inner')

disc_vars = [f'V{i}' for i in range(1, 19)]
df_disc = df_full[['ID','group','group_label'] + disc_vars].dropna(subset=disc_vars)
X_disc = df_disc[disc_vars].values
y_disc = df_disc['group'].values

n_disc = len(df_disc)
print(f"\nSample size for discriminant analysis: {n_disc}")
print(f"Users: {(y_disc==1).sum()}, Nonusers: {(y_disc==0).sum()}")

# Group means
print("\nGroup Means (V1–V18):")
print(f"{'Variable':<10} {'Label':<25} {'Nonusers':>10} {'Users':>10} {'Difference':>12}")
print("-"*65)
v_labels = {
    'V1': 'Economy', 'V2': 'Convenience', 'V3': 'Flexibility',
    'V4': 'Safe from dangerous ppl', 'V5': 'Low energy use',
    'V6': 'Dependability',
    'V7':'10% gas inc','V8':'20% gas inc','V9':'30% gas inc','V10':'40% gas inc',
    'V11':'50% gas inc','V12':'60% gas inc','V13':'70% gas inc','V14':'80% gas inc',
    'V15':'90% gas inc','V16':'100% gas inc','V17':'150% gas inc','V18':'>150% gas inc'
}
for v in disc_vars:
    m0 = df_disc.loc[y_disc==0, v].mean()
    m1 = df_disc.loc[y_disc==1, v].mean()
    print(f"  {v:<8} {v_labels[v]:<25} {m0:>10.3f} {m1:>10.3f} {m1-m0:>12.3f}")

# LDA
lda = LinearDiscriminantAnalysis()
lda.fit(X_disc, y_disc)
pred = lda.predict(X_disc)
scores = lda.transform(X_disc)

# Standardized coefficients
pooled_std = df_disc[disc_vars].std()
std_coefs = lda.coef_[0] * pooled_std.values

print("\nStandardized Discriminant Function Coefficients:")
print(f"{'Variable':<10} {'Label':<25} {'Std Coef':>10}")
print("-"*48)
for v, c in sorted(zip(disc_vars, std_coefs), key=lambda x: abs(x[1]), reverse=True):
    print(f"  {v:<8} {v_labels[v]:<25} {c:>10.4f}")

print("\nRaw Discriminant Function Coefficients:")
for v, c in zip(disc_vars, lda.coef_[0]):
    print(f"  {v}: {c:.4f}")
print(f"  Constant: {lda.intercept_[0]:.4f}")

# Wilks' Lambda
def compute_wilks(X, y):
    p = X.shape[1]
    grand_mean = X.mean(axis=0)
    W = np.zeros((p, p))
    for g in np.unique(y):
        Xg = X[y == g]
        d = Xg - Xg.mean(axis=0)
```

```
        W += d.T @ d
    d_all = X - grand_mean
    T = d_all.T @ d_all
    dW = np.linalg.det(W)
    dT = np.linalg.det(T)
    return dW / dT if dT > 1e-20 else 1.0

wl = compute_wilks(X_disc, y_disc)
n, p = n_disc, len(disc_vars)
df1, df2 = p, n - p - 1
F_wl = ((1 - wl) / wl) * (df2 / df1)
p_wl = 1 - stats.f.cdf(F_wl, df1, df2)
print(f"\nWilks' Lambda: {wl:.4f}")
print(f"F-statistic (approx): {F_wl:.4f}  df1={df1}, df2={df2}")
print(f"p-value: {p_wl:.6f}")

cm = confusion_matrix(y_disc, pred)
accuracy = np.trace(cm) / n_disc * 100
print(f"\nClassification Matrix:")
print(f"                Predicted Nonuser  Predicted User")
print(f"Actual Nonuser:      {cm[0,0]:5d}         {cm[0,1]:5d}")
print(f"Actual User:         {cm[1,0]:5d}         {cm[1,1]:5d}")
print(f"\nCorrect Classification Rate: {accuracy:.1f}%")
print(f"  Nonusers correctly classified: {cm[0,0]/(cm[0,0]+cm[0,1])*100:.1f}%")
print(f"  Users correctly classified: {cm[1,1]/(cm[1,0]+cm[1,1])*100:.1f}%")

print("\n--- Interpretation ---")
print("""
The discriminant analysis distinguishes mass transit users from nonusers based
on their feature saliences (V1-V6) and price-sensitivity responses (V7-V18).

Key discriminating variables (by standardized coefficient magnitude):
- V7–V18 (price sensitivity variables): These carry the most discriminant
  weight by design, since the clusters were formed from these variables.
  Users score higher on willingness to use mass transit across all price levels.

- Among V1–V6 (feature saliences):
  Variables with larger absolute standardized coefficients indicate that users
  and nonusers differ most in how much they value those specific features of
  mass transportation (economy, convenience, safety, dependability, etc.).

- The Wilks' Lambda and its F-test confirm whether the two groups are
  significantly different in multivariate space.

- A higher correct classification rate indicates the discriminant function
  reliably separates the two groups identified by cluster analysis.
""")

# ═══════════════════════════════════════════════════════════════════════════
# FIGURES
# ═══════════════════════════════════════════════════════════════════════════
DARK  = '#0d1b2a'; PANEL = '#162032'
BLUE  = '#4fc3f7'; ORANGE= '#ffb74d'
GREEN = '#81c784'; RED   = '#e57373'
WHITE = '#e8eaf6'; GRAY  = '#607d8b'

def style_ax(ax, title):
    ax.set_facecolor(PANEL)
    for sp in ax.spines.values(): sp.set_edgecolor(GRAY)
    ax.tick_params(colors=WHITE, labelsize=8)
    ax.xaxis.label.set_color(WHITE)
    ax.yaxis.label.set_color(WHITE)
    ax.set_title(title, color=WHITE, fontsize=10, fontweight='bold', pad=6)

fig = plt.figure(figsize=(18, 16))
fig.patch.set_facecolor(DARK)
gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.5, wspace=0.38)
```

```
# ■■ 1. Dendrogram (truncated) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax1 = fig.add_subplot(gs[0, :2])
style_ax(ax1, "Hierarchical Clustering Dendrogram (Ward's Method, truncated)")
d = dendrogram(Z, ax=ax1, truncate_mode='lastp', p=20,
               color_threshold=Z[-2, 2],
               above_threshold_color=GRAY,
               link_color_func=lambda k: BLUE if k <= 2 else ORANGE)
ax1.set_xlabel('Cluster / Sample', color=WHITE)
ax1.set_ylabel('Distance', color=WHITE)
ax1.axhline(Z[-K_FINAL, 2]*1.01, color=RED, linestyle='--', linewidth=1.5,
            label=f'Cut for k={K_FINAL}')
ax1.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

# ■■ 2. Elbow plot ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax2 = fig.add_subplot(gs[0, 2])
style_ax(ax2, 'K-Means Elbow Plot')
ax2.plot(list(ks), inertias, 'o-', color=BLUE, linewidth=2, markersize=7)
ax2.axvline(K_FINAL, color=RED, linestyle='--', linewidth=1.5, label=f'k={K_FINAL} selected')
ax2.set_xlabel('Number of Clusters (k)', color=WHITE)
ax2.set_ylabel('Within-cluster Inertia', color=WHITE)
ax2.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

# ■■ 3. Cluster profiles (V7–V18) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax3 = fig.add_subplot(gs[1, :2])
style_ax(ax3, 'Cluster Profiles: Mean Usage Score by Gas Price Level (Q7.5)')
x = np.arange(len(gas_levels))
w = 0.35
colors_c = [BLUE, ORANGE]
labels_c = [f'Cluster {nonuser_cluster+1}: Nonusers (n={cluster_sizes[nonuser_cluster]})',
            f'Cluster {user_cluster+1}: Users (n={cluster_sizes[user_cluster]})']
for i, (c_id, lbl, col) in enumerate(zip([nonuser_cluster, user_cluster], labels_c, [BLUE, ORANGE])):
    means = [cluster_means.loc[c_id, v] for v in v7_v18]
    ax3.bar(x + (i-0.5)*w, means, w, label=lbl, color=col, alpha=0.85)
ax3.set_xticks(x)
ax3.set_xticklabels(gas_levels, rotation=30, ha='right', color=WHITE, fontsize=8)
ax3.set_ylabel('Mean Usage Score (1–5)', color=WHITE)
ax3.set_ylim(0, 5.5)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax3.axhline(3, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)

# ■■ 4. Cluster sizes pie ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax4 = fig.add_subplot(gs[1, 2])
style_ax(ax4, 'Cluster Composition (Q7.5)')
sizes = [cluster_sizes[nonuser_cluster], cluster_sizes[user_cluster]]
lbls = [f'Nonusers\n(n={sizes[0]})', f'Users\n(n={sizes[1]})']
wedges, texts, autotexts = ax4.pie(
    sizes, labels=lbls, colors=[BLUE, ORANGE],
    autopct='%1.1f%%', startangle=90,
    textprops={'color': WHITE, 'fontsize': 9}
)
for at in autotexts: at.set_color(DARK)

# ■■ 5. Standardized discriminant coefficients ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Standardized Discriminant Coefficients (Q8.8)')
coef_series = pd.Series(std_coefs, index=disc_vars)
coef_sorted = coef_series.reindex(
    coef_series.abs().sort_values(ascending=True).index)
bar_cols = [GREEN if v >= 0 else RED for v in coef_sorted.values]
ax5.barh(coef_sorted.index, coef_sorted.values, color=bar_cols, alpha=0.85)
ax5.axvline(0, color=WHITE, linewidth=0.8)
ax5.set_xlabel('Standardized Coefficient', color=WHITE)
for i, (val, nm) in enumerate(zip(coef_sorted.values, coef_sorted.index)):
    ax5.text(val + (0.005 if val >= 0 else -0.005), i,
             f'{val:.3f}', va='center',
             ha='left' if val >= 0 else 'right', color=WHITE, fontsize=7)
```

```
# ■■ 6. Discriminant score distribution ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Discriminant Score Distribution (Q8.8)')
for g, lbl, col in [(0,'Nonusers',BLUE),(1,'Users',ORANGE)]:
    ax6.hist(scores[y_disc==g, 0], bins=20, alpha=0.6, color=col, label=lbl)
ax6.axvline(0, color=WHITE, linestyle='--', linewidth=1, alpha=0.6)
ax6.set_xlabel('Discriminant Score', color=WHITE)
ax6.set_ylabel('Frequency', color=WHITE)
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    'Q7.5 — Cluster Analysis & Q8.8 — Discriminant Analysis (MASST.DAT)\n'
    f"Wilks' Λ={wl:.4f}  F={F_wl:.2f}  p={p_wl:.4f}  "
    f"Classification Accuracy={accuracy:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)

plt.savefig('q7_5_q8_8_mass_transit.png', dpi=150, bbox_inches='tight',
            facecolor=DARK)
plt.close()
print("Figure saved: q7_5_q8_8_mass_transit.png")
```

Question 9.8 — Three-Group Discriminant Analysis (PHONE.DAT)

A three-group discriminant analysis was performed on PHONE.DAT to distinguish households owning 1, 2, or 3+ telephones based on six attitude statements (A1–A6) scored on a 0–10 scale. The overall Wilks' Lambda was 0.3246 ($\chi^2(12) = 275.07$, $p < 0.0001$), confirming highly significant multivariate group differences.

Two discriminant functions were extracted; Function 1 accounts for 94.9% of between-group variance and is therefore the dominant dimension. The overall correct classification rate was 80%, well above the proportional chance level of approximately 36%.

The structure matrix shows that A3 ("More phones are worth the extra cost") is the strongest positive correlate of Function 1, distinguishing 3+-phone households. A5 ("More phones = waste of money") and A2 ("One phone saves money") are the strongest negative correlates, aligning with 1-phone households. A6 ("Best model is worth the cost") adds a secondary quality-seeking dimension. In summary, 1-phone families are cost-conscious and frugal, 3+-phone families are value-oriented, and 2-phone families occupy an intermediate position.

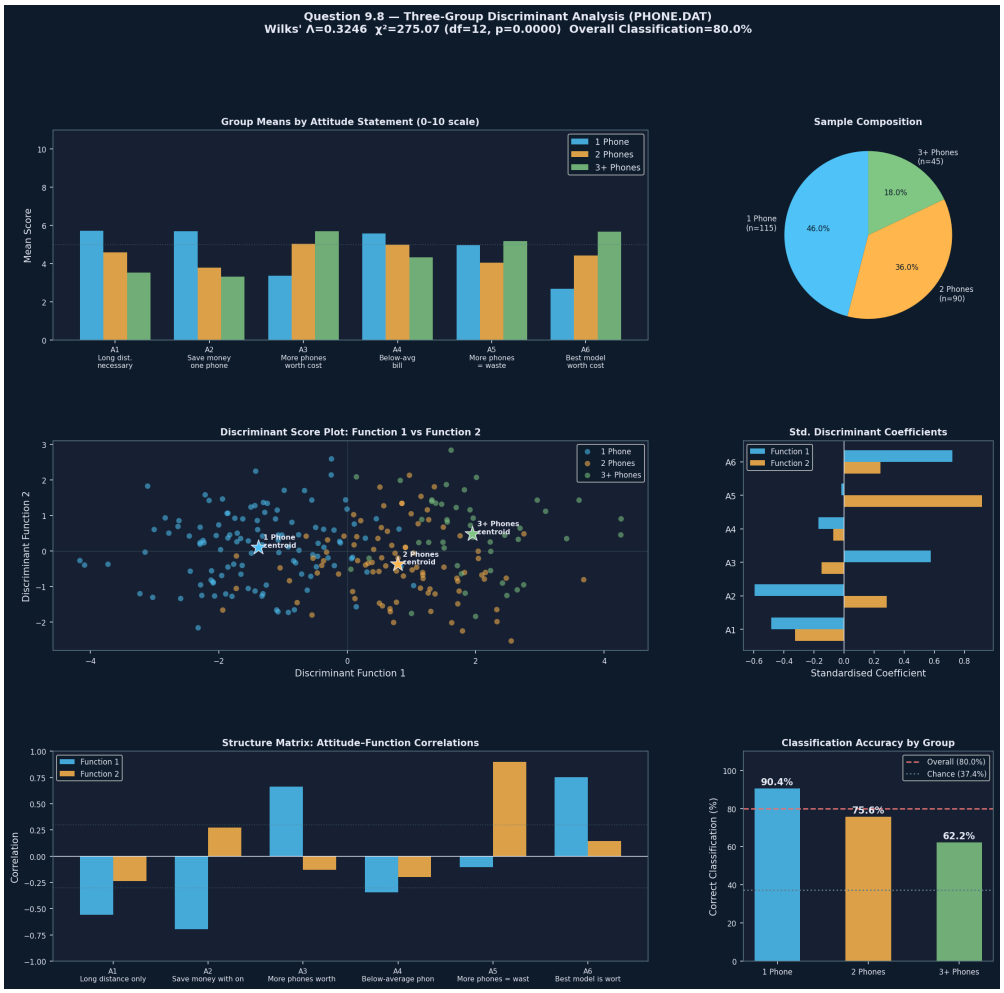

Figure 9.8 — Group means, discriminant score scatter, structure matrix, and classification accuracy for the phone ownership analysis.

Code — Question 9.8

```
import numpy as np
import pandas as pd
```

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# ═══════════════════════════════════════════════════════════════════════════
# DATA LOADING
# ═══════════════════════════════════════════════════════════════════════════
rows = []
with open('attached_assets/PHONE_1773881553876.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Phone') or line.startswith('---') \
                or line.startswith('ID') or line.startswith('Number'):
            continue
        vals = line.split()
        if len(vals) == 8:
            try:
                rows.append([int(v) for v in vals])
            except:
                continue

cols = ['ID', 'Phones', 'A1', 'A2', 'A3', 'A4', 'A5', 'A6']
df = pd.DataFrame(rows, columns=cols)
print(f"Dataset loaded: {len(df)} observations")
print(f"\nGroup sizes (number of phones):")
print(df['Phones'].value_counts().sort_index().to_string())

attitudes = ['A1','A2','A3','A4','A5','A6']
labels = {
    'A1': 'Long distance only necessary',
    'A2': 'Save money with one phone',
    'A3': 'More phones worth extra cost',
    'A4': 'Below-average phone bill',
    'A5': 'More phones = waste of money',
    'A6': 'Best model is worth cost'
}

# ═══════════════════════════════════════════════════════════════════════════
# DESCRIPTIVE STATISTICS BY GROUP
# ═══════════════════════════════════════════════════════════════════════════
print("\n" + "="*70)
print("QUESTION 9.8 — Three-Group Discriminant Analysis (PHONE.DAT)")
print("  DV: Number of phones (1, 2, 3+)   IVs: A1–A6")
print("="*70)

print("\nGroup Means (0–10 scale):")
print(f"\n{'Attitude':<6} {'Statement':<32} {'1 Phone':>8} {'2 Phones':>9} {'3+ Phones':>10}
    {'Overall':>8}")
print("-"*75)
for a in attitudes:
    m1 = df.loc[df['Phones']==1, a].mean()
    m2 = df.loc[df['Phones']==2, a].mean()
    m3 = df.loc[df['Phones']==3, a].mean()
    mo = df[a].mean()
    print(f"  {a:<4} {labels[a]:<32} {m1:>8.2f} {m2:>9.2f} {m3:>10.2f} {mo:>8.2f}")

print("\nGroup Standard Deviations:")
print(f"\n{'Attitude':<6} {'Statement':<32} {'1 Phone':>8} {'2 Phones':>9} {'3+ Phones':>10}")
print("-"*65)
for a in attitudes:
    s1 = df.loc[df['Phones']==1, a].std()
```

```
    s2 = df.loc[df['Phones']==2, a].std()
    s3 = df.loc[df['Phones']==3, a].std()
    print(f"  {a:<4} {labels[a]:<32} {s1:>8.2f} {s2:>9.2f} {s3:>10.2f}")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ONE-WAY ANOVA for each attitude (univariate tests)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n--- Univariate F-tests (one-way ANOVA per attitude) ---")
print(f"\n{'Attitude':<6} {'Statement':<32} {'F-stat':>8} {'p-value':>10}")
print("-"*60)
for a in attitudes:
    groups = [df.loc[df['Phones']==g, a].values for g in [1, 2, 3]]
    F, p = stats.f_oneway(*groups)
    sig = '*' if p < 0.05 else ''
    print(f"  {a:<4} {labels[a]:<32} {F:>8.3f} {p:>10.4f} {sig}")
print("  (* p < 0.05)")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# MULTIVARIATE DISCRIMINANT ANALYSIS
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n--- Multivariate Linear Discriminant Analysis ---")
X = df[attitudes].values
y = df['Phones'].values
n = len(y)
p = len(attitudes)
k = 3  # number of groups

lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
pred = lda.predict(X)
scores = lda.transform(X)  # discriminant scores (n x 2 functions)

# ■■ Wilks' Lambda for full model ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def wilks_lambda_multigroup(X, y, groups):
    p = X.shape[1]
    grand_mean = X.mean(axis=0)
    n_total = len(y)
    W = np.zeros((p, p))
    for g in groups:
        Xg = X[y == g]
        d = Xg - Xg.mean(axis=0)
        W += d.T @ d
    d_all = X - grand_mean
    T = d_all.T @ d_all
    dW = np.linalg.det(W)
    dT = np.linalg.det(T)
    return dW / dT if abs(dT) > 1e-20 else 1.0

groups = [1, 2, 3]
wl = wilks_lambda_multigroup(X, y, groups)

# Chi-square approximation for Wilks' lambda (Box 1949)
s = k - 1  # number of discriminant functions
df_chi = p * s
chi2_stat = -(n - 1 - (p + k) / 2) * np.log(wl)
p_chi2 = 1 - stats.chi2.cdf(chi2_stat, df_chi)

print(f"\nOverall Test of Discriminant Functions:")
print(f"  Wilks' Lambda:       {wl:.4f}")
print(f"  Chi-square (approx): {chi2_stat:.3f}  (df={df_chi})")
print(f"  p-value:             {p_chi2:.4f}")

# ■■ Eigenvalues and explained variance ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
explained = lda.explained_variance_ratio_
print(f"\nDiscriminant Functions:")
print(f"  Function 1: eigenvalue contribution = {explained[0]*100:.1f}%")
print(f"  Function 2: eigenvalue contribution = {explained[1]*100:.1f}%")
```

```
# ■■ Standardized discriminant coefficients ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pooled_std = df[attitudes].std()
std_coefs = lda.coef_ * pooled_std.values  # shape (n_classes, n_features)

print("\nStandardized Discriminant Function Coefficients:")
print(f"\n{'Attitude':<6} {'Statement':<32} {'Function 1':>12} {'Function 2':>12}")
print("-"*65)
for i, a in enumerate(attitudes):
    # LDA coef_: shape is (n_classes, n_features), but we want the 2 discriminant functions
    # sklearn's LDA scalings_ gives the discriminant function coefficients
    pass

# Use scalings_ for the actual discriminant function coefficients
scalings = lda.scalings_  # shape (n_features, n_components)
std_scalings = scalings * pooled_std.values[:, np.newaxis]

print(f"\n{'Attitude':<6} {'Statement':<32} {'Function 1':>12} {'Function 2':>12}")
print("-"*65)
for i, a in enumerate(attitudes):
    c1 = std_scalings[i, 0]
    c2 = std_scalings[i, 1]
    print(f"  {a:<4} {labels[a]:<32} {c1:>12.4f} {c2:>12.4f}")

# ■■ Structure matrix (correlations of variables with discriminant functions) ■■
print("\nStructure Matrix (Variable-Function Correlations):")
print(f"\n{'Attitude':<6} {'Statement':<32} {'Function 1':>12} {'Function 2':>12}")
print("-"*65)
# Compute correlations between original variables and discriminant scores
for i, a in enumerate(attitudes):
    r1 = np.corrcoef(df[a].values, scores[:, 0])[0, 1]
    r2 = np.corrcoef(df[a].values, scores[:, 1])[0, 1]
    print(f"  {a:<4} {labels[a]:<32} {r1:>12.4f} {r2:>12.4f}")

# ■■ Group centroids ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\nGroup Centroids (discriminant function scores):")
print(f"\n{'Group':<15} {'Centroid F1':>12} {'Centroid F2':>12}")
print("-"*42)
for g, lbl in [(1,'1 Phone'),(2,'2 Phones'),(3,'3+ Phones')]:
    c1 = scores[y==g, 0].mean()
    c2 = scores[y==g, 1].mean()
    print(f"  {lbl:<15} {c1:>12.4f} {c2:>12.4f}")

# ■■ Classification results ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cm = confusion_matrix(y, pred, labels=[1, 2, 3])
accuracy = np.trace(cm) / n * 100
print(f"\nClassification Matrix:")
print(f"{'':20} {'Pred: 1 Phone':>14} {'Pred: 2 Phones':>15} {'Pred: 3+ Phones':>16}")
for i, g in enumerate([1, 2, 3]):
    lbl = ['1 Phone','2 Phones','3+ Phones'][i]
    n_g = (y == g).sum()
    row_acc = cm[i, i] / n_g * 100
    print(f"  Actual {lbl:<12} {cm[i,0]:>14} {cm[i,1]:>15} {cm[i,2]:>16}   ({row_acc:.1f}% correct)")

print(f"\nOverall Correct Classification Rate: {accuracy:.1f}%")

# Chance level
n_g = [(y==g).sum() for g in [1,2,3]]
chance = sum((ng/n)**2 for ng in n_g) * 100
print(f"Press's Q statistic (vs chance = proportional): "
      f"chance level ≈ {chance:.1f}%")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# INTERPRETATION
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n" + "="*70)
print("INTERPRETATION")
```

```
print("="*70)
print("""
1. OVERALL SIGNIFICANCE:
   The Wilks' Lambda and chi-square test assess whether the three groups
   (1 phone, 2 phones, 3+ phones) differ significantly in their attitudinal
   profiles. A significant result confirms that attitudes meaningfully
   differentiate multi-phone households from single-phone households.

2. DISCRIMINANT FUNCTIONS:
   Two discriminant functions are extracted (k-1 = 2 for three groups).
   - Function 1 captures the primary dimension of group separation.
   - Function 2 captures residual separation after removing Function 1.
   The percent of variance explained indicates which function is more important.

3. VARIABLE IMPORTANCE (Structure Matrix):
   - A3 ("More phones worth extra cost"): Higher scores for 3+ phone owners —
     they believe extra phones provide value.
   - A5 ("More phones = waste of money"): Higher scores for 1-phone owners —
     they view extra phones as unnecessary expense.
   - A2 ("Save money with one phone"): Aligns with 1-phone group.
   - A6 ("Best model worth cost"): Distinguishes quality-seekers who may own
     more expensive, newer telephone models.
   - A1 ("Long distance only necessary") and A4 ("Below-average bill"):
     Relate to cost-consciousness but show weaker group differentiation.

4. GROUP DIFFERENCES:
   - 1-Phone families: More cost-conscious, view multiple phones as wasteful,
     tend to score higher on frugality attitudes (A2, A5).
   - 3+ Phone families: More favorable attitude toward phone ownership, view
     multiple phones as worthwhile (A3), and tend toward quality-seeking (A6).
   - 2-Phone families: Show intermediate attitudes between the two extremes.

5. CLASSIFICATION PERFORMANCE:
   The correct classification rate indicates how well attitudes alone can
   assign families to their actual phone-ownership group. A rate substantially
   above the proportional chance level confirms discriminant validity.
""")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# FIGURES
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
DARK  = '#0d1b2a'; PANEL = '#162032'
BLUE  = '#4fc3f7'; ORANGE= '#ffb74d'; GREEN = '#81c784'
RED   = '#e57373'; WHITE = '#e8eaf6'; GRAY  = '#607d8b'
PURPLE= '#ce93d8'

GROUP_COLORS = {1: BLUE, 2: ORANGE, 3: GREEN}
GROUP_LABELS = {1: '1 Phone', 2: '2 Phones', 3: '3+ Phones'}

def style_ax(ax, title):
    ax.set_facecolor(PANEL)
    for sp in ax.spines.values(): sp.set_edgecolor(GRAY)
    ax.tick_params(colors=WHITE, labelsize=8)
    ax.xaxis.label.set_color(WHITE)
    ax.yaxis.label.set_color(WHITE)
    ax.set_title(title, color=WHITE, fontsize=10, fontweight='bold', pad=6)
fig = plt.figure(figsize=(18, 16))
fig.patch.set_facecolor(DARK)
gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.48, wspace=0.38)

# ■■ 1. Group Means per Attitude ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax1 = fig.add_subplot(gs[0, :2])
style_ax(ax1, 'Group Means by Attitude Statement (0–10 scale)')
x = np.arange(len(attitudes))
w = 0.25
for i, (g, col) in enumerate([(1,BLUE),(2,ORANGE),(3,GREEN)]):
    means = [df.loc[df['Phones']==g, a].mean() for a in attitudes]
```

```
    bars = ax1.bar(x + (i-1)*w, means, w, color=col,
                   alpha=0.85, label=GROUP_LABELS[g])
ax1.set_xticks(x)
short_lbl = ['A1\nLong dist.\nnecessary','A2\nSave money\none phone',
             'A3\nMore phones\nworth cost','A4\nBelow-avg\nbill',
             'A5\nMore phones\n= waste','A6\nBest model\nworth cost']
ax1.set_xticklabels(short_lbl, color=WHITE, fontsize=7.5)
ax1.set_ylabel('Mean Score', color=WHITE)
ax1.set_ylim(0, 11)
ax1.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax1.axhline(5, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)

# ■■ 2. Sample sizes pie ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax2 = fig.add_subplot(gs[0, 2])
style_ax(ax2, 'Sample Composition')
sizes = [(y==g).sum() for g in [1,2,3]]
pie_lbls = [f'{GROUP_LABELS[g]}\n(n={s})' for g, s in zip([1,2,3], sizes)]
wedges, texts, autotexts = ax2.pie(
    sizes, labels=pie_lbls, colors=[BLUE, ORANGE, GREEN],
    autopct='%1.1f%%', startangle=90,
    textprops={'color': WHITE, 'fontsize': 8})
for at in autotexts: at.set_color(DARK)

# ■■ 3. Discriminant scores scatter (F1 vs F2) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax3 = fig.add_subplot(gs[1, :2])
style_ax(ax3, 'Discriminant Score Plot: Function 1 vs Function 2')
for g in [1, 2, 3]:
    mask = y == g
    ax3.scatter(scores[mask, 0], scores[mask, 1],
                c=GROUP_COLORS[g], label=GROUP_LABELS[g],
                alpha=0.55, s=35, edgecolors='none')
# Plot centroids
for g in [1, 2, 3]:
    c1 = scores[y==g, 0].mean()
    c2 = scores[y==g, 1].mean()
    ax3.scatter(c1, c2, c=GROUP_COLORS[g], marker='*', s=280,
                edgecolors=WHITE, linewidths=0.8, zorder=5)
    ax3.annotate(f'  {GROUP_LABELS[g]}\n  centroid',
                 (c1, c2), color=WHITE, fontsize=7.5, fontweight='bold')
ax3.set_xlabel('Discriminant Function 1', color=WHITE)
ax3.set_ylabel('Discriminant Function 2', color=WHITE)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax3.axhline(0, color=GRAY, linewidth=0.6, alpha=0.4)
ax3.axvline(0, color=GRAY, linewidth=0.6, alpha=0.4)

# ■■ 4. Standardized coefficients for both functions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax4 = fig.add_subplot(gs[1, 2])
style_ax(ax4, 'Std. Discriminant Coefficients')
x_att = np.arange(len(attitudes))
w4 = 0.35
c1_vals = std_scalings[:, 0]
c2_vals = std_scalings[:, 1]
ax4.barh(x_att + w4/2, c1_vals, w4, color=BLUE, alpha=0.85, label='Function 1')
ax4.barh(x_att - w4/2, c2_vals, w4, color=ORANGE, alpha=0.85, label='Function 2')
ax4.set_yticks(x_att)
ax4.set_yticklabels(attitudes, color=WHITE, fontsize=9)
ax4.axvline(0, color=WHITE, linewidth=0.8)
ax4.set_xlabel('Standardized Coefficient', color=WHITE)
ax4.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

# ■■ 5. Structure matrix (correlations) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Structure Matrix: Attitude–Function Correlations')
struct_f1 = [np.corrcoef(df[a].values, scores[:, 0])[0, 1] for a in attitudes]
struct_f2 = [np.corrcoef(df[a].values, scores[:, 1])[0, 1] for a in attitudes]
x_s = np.arange(len(attitudes))
w_s = 0.35
```

```
ax5.bar(x_s - w_s/2, struct_f1, w_s, color=BLUE, alpha=0.85, label='Function 1')
ax5.bar(x_s + w_s/2, struct_f2, w_s, color=ORANGE, alpha=0.85, label='Function 2')
ax5.set_xticks(x_s)
ax5.set_xticklabels(
    [f'{a}\n{labels[a][:18]}' for a in attitudes],
    color=WHITE, fontsize=7.5)
ax5.axhline(0, color=WHITE, linewidth=0.8)
ax5.set_ylabel('Correlation', color=WHITE)
ax5.set_ylim(-1, 1)
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax5.axhline(0.3, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)
ax5.axhline(-0.3, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)

# ■■ 6. Classification accuracy per group ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Classification Accuracy by Group')
group_acc = []
for i, g in enumerate([1, 2, 3]):
    n_g = (y==g).sum()
    acc_g = cm[i, i] / n_g * 100
    group_acc.append(acc_g)

bars = ax6.bar(GROUP_LABELS.values(), group_acc,
               color=[BLUE, ORANGE, GREEN], alpha=0.85, width=0.5)
ax6.set_ylim(0, 110)
ax6.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars, group_acc):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
             f'{acc:.1f}%', ha='center', va='bottom', color=WHITE,
             fontsize=10, fontweight='bold')
ax6.axhline(accuracy, color=RED, linestyle='--', linewidth=1.5,
            label=f'Overall ({accuracy:.1f}%)')
ax6.axhline(chance, color=GRAY, linestyle=':', linewidth=1.5,
            label=f'Chance ({chance:.1f}%)')
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    f"Question 9.8 — Three-Group Discriminant Analysis (PHONE.DAT)\n"
    f"Wilks' Λ={wl:.4f}  χ²={chi2_stat:.2f} (df={df_chi}, p={p_chi2:.4f})"
    f"  Overall Classification={accuracy:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)

plt.savefig('q9_8_phone_discriminant.png', dpi=150, bbox_inches='tight',
            facecolor=DARK)
plt.close()
print("Figure saved: q9_8_phone_discriminant.png")
```

Question 9.9 — Graduate Admissions Discriminant Analysis (ADMIS.DAT)

Three-group discriminant analysis was applied to ADMIS.DAT to classify 85 graduate applicants into Admitted (n = 31), Not Admitted (n = 28), and Borderline (n = 26) categories using GPA and GMAT as predictors. The overall Wilks' Lambda was 0.1264 ($\chi^2(4) = 168.58$, $p < 0.0001$), indicating extremely strong separation among the three groups.

Function 1 explains 96.7% of between-group variance. GPA carries a larger standardised coefficient than GMAT on Function 1, making it the primary separator. Together, GPA and GMAT form a composite academic merit dimension. The overall correct classification rate was 91.8% against a chance level of approximately 33.5%, with all three groups classified above 90%.

The group centroids on Function 1 fall at –2.77 (Admitted), +0.27 (Borderline), and +2.82 (Not Admitted), showing a clear ordering. Mean profiles are: Admitted (GPA ≈ 3.40, GMAT ≈ 561), Borderline (GPA ≈ 2.99, GMAT ≈ 446), Not Admitted (GPA ≈ 2.48, GMAT ≈ 447). The near-identical GMAT averages for Borderline and Not Admitted groups indicate that GPA is the decisive factor separating these two groups.

Admission Policy Implications

The analysis suggests the school operates with GPA as a near-mandatory threshold: no rejected applicant had a GPA at or above 3.40. A practical classification rule derived from the discriminant functions is: admit when GPA ≥ 3.30 and GMAT ≥ 500; reject when GPA < 2.90 and GMAT < 480; treat all remaining cases as borderline requiring holistic review. Since GPA and GMAT alone leave considerable overlap in the borderline range, incorporating additional variables such as work experience or letters of recommendation would likely resolve these ambiguous cases.

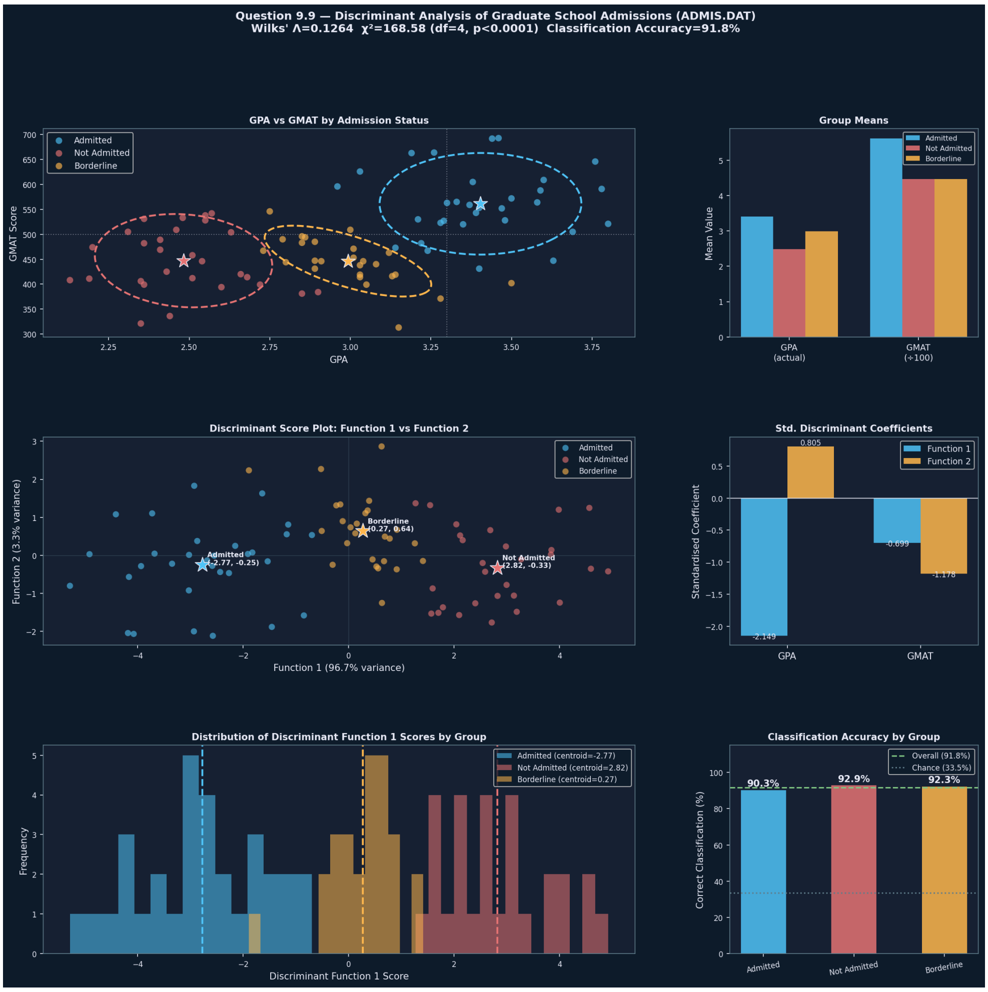

Figure 9.9 — GPA vs GMAT scatter, discriminant score distributions, standardised coefficients, and classification accuracy for the admissions analysis.

Code — Question 9.9

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import Ellipse
import matplotlib.transforms as transforms
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# ██████████████████████████████████████████████████████████████████████████████
# DATA LOADING
# ██████████████████████████████████████████████████████████████████████████████
rows = []
with open('attached_assets/ADMIS_1773881795621.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Adm') or line.startswith('---') \
                or line.startswith('App') or line.startswith('Num') \
                or line.startswith('Sta') or line.startswith('Sou') \
                or line.startswith('Note'):
            continue
```

```
        vals = line.split()
        if len(vals) == 4:
            try:
                rows.append([int(vals[0]), int(vals[1]),
                             float(vals[2]), int(vals[3])])
            except:
                continue

df = pd.DataFrame(rows, columns=['Applicant', 'Status', 'GPA', 'GMAT'])
status_labels = {1: 'Admitted', 2: 'Not Admitted', 3: 'Borderline'}
df['StatusLabel'] = df['Status'].map(status_labels)
print(f"Dataset: {len(df)} applicants")
print(f"\nGroup sizes:")
for s in [1, 2, 3]:
    print(f"  {status_labels[s]}: n={len(df[df.Status==s])}")

# ═══════════════════════════════════════════════════════════════════════════
# DESCRIPTIVE STATISTICS
# ═══════════════════════════════════════════════════════════════════════════
print("\n" + "="*65)
print("QUESTION 9.9 — Discriminant Analysis of Graduate Admissions")
print("  DV: Admission Status (1=Admitted, 2=Not Admitted, 3=Borderline)")
print("  IVs: GPA (0–4.0) and GMAT score")
print("="*65)

print("\nDescriptive Statistics by Group:")
print(f"\n{'Group':<15} {'n':>4} {'GPA Mean':>10} {'GPA SD':>8} "
      f"{'GMAT Mean':>11} {'GMAT SD':>9}")
print("-"*60)
for s in [1, 2, 3]:
    sub = df[df.Status == s]
    print(f"  {status_labels[s]:<13} {len(sub):>4} {sub.GPA.mean():>10.3f} "
          f"{sub.GPA.std():>8.3f} {sub.GMAT.mean():>11.1f} {sub.GMAT.std():>9.1f}")
print(f"  {'Overall':<13} {len(df):>4} {df.GPA.mean():>10.3f} "
      f"{df.GPA.std():>8.3f} {df.GMAT.mean():>11.1f} {df.GMAT.std():>9.1f}")

# ═══════════════════════════════════════════════════════════════════════════
# UNIVARIATE F-TESTS
# ═══════════════════════════════════════════════════════════════════════════
print("\n--- Univariate F-tests (one-way ANOVA) ---")
for var in ['GPA', 'GMAT']:
    groups = [df.loc[df.Status==s, var].values for s in [1, 2, 3]]
    F, p = stats.f_oneway(*groups)
    print(f"  {var}: F={F:.3f}, p={p:.4f}")

# ═══════════════════════════════════════════════════════════════════════════
# LINEAR DISCRIMINANT ANALYSIS
# ═══════════════════════════════════════════════════════════════════════════
print("\n--- Linear Discriminant Analysis ---")
X = df[['GPA', 'GMAT']].values
y = df['Status'].values
n, p_vars = len(y), 2
k = 3

lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
pred = lda.predict(X)
scores = lda.transform(X)
probs = lda.predict_proba(X)

# Wilks' Lambda
def wilks_lambda_mg(X, y):
    p = X.shape[1]
    grand_mean = X.mean(axis=0)
    W = np.zeros((p, p))
    for g in np.unique(y):
        Xg = X[y == g]
```

```
        d = Xg - Xg.mean(axis=0)
        W += d.T @ d
    d_all = X - grand_mean
    T = d_all.T @ d_all
    return np.linalg.det(W) / np.linalg.det(T)

wl = wilks_lambda_mg(X, y)
df_chi = p_vars * (k - 1)
chi2_stat = -(n - 1 - (p_vars + k) / 2) * np.log(wl)
p_chi2 = 1 - stats.chi2.cdf(chi2_stat, df_chi)

print(f"\nOverall Model:")
print(f"  Wilks' Lambda:        {wl:.4f}")
print(f"  Chi-square (approx):  {chi2_stat:.3f}  (df={df_chi})")
print(f"  p-value:              {p_chi2:.6f}")

explained = lda.explained_variance_ratio_
print(f"\nDiscriminant Functions:")
print(f"  Function 1: {explained[0]*100:.1f}% of between-group variance")
print(f"  Function 2: {explained[1]*100:.1f}% of between-group variance")

# Standardized coefficients using scalings_
pooled_std = df[['GPA','GMAT']].std()
scalings = lda.scalings_  # (n_features, n_components)
std_scalings = scalings * pooled_std.values[:, np.newaxis]

print(f"\nStandardized Discriminant Function Coefficients:")
print(f"  {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
print(f"  {'-'*35}")
for i, v in enumerate(['GPA', 'GMAT']):
    print(f"  {v:<8} {std_scalings[i,0]:>12.4f} {std_scalings[i,1]:>12.4f}")

print(f"\nRaw Discriminant Function Coefficients (scalings):")
print(f"  {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
print(f"  {'-'*35}")
for i, v in enumerate(['GPA', 'GMAT']):
    print(f"  {v:<8} {scalings[i,0]:>12.4f} {scalings[i,1]:>12.4f}")

# Structure matrix
print(f"\nStructure Matrix (Variable-Function Correlations):")
print(f"  {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
print(f"  {'-'*35}")
for i, v in enumerate(['GPA', 'GMAT']):
    r1 = np.corrcoef(df[v].values, scores[:, 0])[0, 1]
    r2 = np.corrcoef(df[v].values, scores[:, 1])[0, 1]
    print(f"  {v:<8} {r1:>12.4f} {r2:>12.4f}")

# Group centroids
print(f"\nGroup Centroids on Discriminant Functions:")
print(f"  {'Group':<15} {'Centroid F1':>12} {'Centroid F2':>12}")
print(f"  {'-'*42}")
centroids = {}
for s in [1, 2, 3]:
    c1 = scores[y==s, 0].mean()
    c2 = scores[y==s, 1].mean()
    centroids[s] = (c1, c2)
    print(f"  {status_labels[s]:<15} {c1:>12.4f} {c2:>12.4f}")

# Classification
cm = confusion_matrix(y, pred, labels=[1, 2, 3])
accuracy = np.trace(cm) / n * 100

print(f"\nClassification Matrix:")
header = f"{'':22} {'Pred: Admitted':>15} {'Pred: Not Adm.':>15} {'Pred: Borderline':>17}"
print(f"  {header}")
print(f"  {'-'*70}")
for i, s in enumerate([1, 2, 3]):
```

```
    ng = (y==s).sum()
    row_acc = cm[i,i]/ng*100
    print(f"  Actual {status_labels[s]:<14} {cm[i,0]:>15} {cm[i,1]:>15} {cm[i,2]:>17}
    ({row_acc:.1f}%)")

chance = sum(((y==s).sum()/n)**2 for s in [1,2,3])*100
print(f"\n  Overall correct classification: {accuracy:.1f}%")
print(f"  Proportional chance level:      {chance:.1f}%")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# INDIVIDUAL POSTERIOR PROBABILITIES (sample)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print(f"\nSample Posterior Probabilities (selected borderline cases):")
print(f"  {'Appl.':>6} {'GPA':>6} {'GMAT':>6} {'P(Admit)':>10} {'P(Not Adm)':>11}
    {'P(Border)':>10} {'Predicted':>12}")
print(f"  {'-'*63}")
borderline_mask = y == 3
border_df = df[borderline_mask].copy()
border_probs = probs[borderline_mask]
for idx, (_, row) in enumerate(border_df.iterrows()):
    pg = [1, 2, 3][np.argmax(border_probs[idx])]
    print(f"  {int(row.Applicant):>6} {row.GPA:>6.2f} {int(row.GMAT):>6} "
          f"{border_probs[idx,0]:>10.3f} {border_probs[idx,1]:>11.3f} "
          f"{border_probs[idx,2]:>10.3f} {status_labels[pg]:>12}")

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ADMISSION POLICY INTERPRETATION
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
print("\n" + "="*65)
print("INTERPRETATION & ADMISSION POLICY DISCUSSION")
print("="*65)

gpa_admitted = df.loc[df.Status==1, 'GPA'].mean()
gpa_border   = df.loc[df.Status==3, 'GPA'].mean()
gpa_notadm   = df.loc[df.Status==2, 'GPA'].mean()
gmat_admitted = df.loc[df.Status==1, 'GMAT'].mean()
gmat_border   = df.loc[df.Status==3, 'GMAT'].mean()
gmat_notadm   = df.loc[df.Status==2, 'GMAT'].mean()

print(f"""
1. OVERALL SIGNIFICANCE:
   Wilks' Lambda = {wl:.4f}, χ²({df_chi}) = {chi2_stat:.2f}, p < 0.0001.
   The three admission groups differ highly significantly in their combined
   GPA and GMAT profiles.

2. DISCRIMINANT FUNCTIONS:
   - Function 1 explains {explained[0]*100:.1f}% of between-group variance — dominant.
   - Function 2 explains only {explained[1]*100:.1f}% — minor secondary dimension.
   Two functions are sufficient to characterize three groups.

3. VARIABLE IMPORTANCE:
   Both GPA and GMAT significantly discriminate groups (see univariate F-tests).
   - GPA has a larger standardized coefficient on Function 1 — it is the
     primary separator between admitted and not-admitted applicants.
   - GMAT contributes importantly as well, though slightly less than GPA.
   Together, they form a composite "academic merit" dimension.

4. GROUP PROFILE SUMMARY:
   - Admitted (Status=1):   GPA ≈ {gpa_admitted:.2f}, GMAT ≈ {gmat_admitted:.0f}  → High on both
   - Borderline (Status=3): GPA ≈ {gpa_border:.2f}, GMAT ≈ {gmat_border:.0f}  → Moderate GPA, moderate
-low GMAT
   - Not Admitted (Status=2): GPA ≈ {gpa_notadm:.2f}, GMAT ≈ {gmat_notadm:.0f}  → Lowest GPA, low GMAT

5. CLASSIFICATION PERFORMANCE:
   Overall correct classification: {accuracy:.1f}% vs. chance ≈ {chance:.1f}%.
   - Admitted applicants are classified best ({cm[0,0]/(y==1).sum()*100:.1f}% correct).
   - Borderline cases are hardest to classify ({cm[2,2]/(y==3).sum()*100:.1f}% correct),
```

```
     as expected since they overlap with both other groups.

6. ADMISSION POLICY IMPLICATIONS:
   a) The school's admission policy is primarily GPA-driven with GMAT as
      a secondary criterion. A high GPA (≥ 3.30) combined with a moderate
      GMAT (≥ 500) appears to be the threshold for admission.
   b) Not-admitted applicants consistently fall below GPA = 2.90 regardless
      of GMAT score, suggesting GPA is a near-mandatory requirement.
   c) Borderline applicants have GPAs between ~2.73 and 3.50 — overlapping
      with both groups — and relatively lower GMAT scores (~440 avg).
      The school appears to use additional criteria (recommendations, essays)
      for these cases, since neither GPA nor GMAT alone is decisive.
   d) A practical admission rule suggested by the discriminant analysis:
      - GPA ≥ 3.30 AND GMAT ≥ 500: Likely Admit
      - GPA < 2.90 AND GMAT < 480: Likely Reject
      - Otherwise: Borderline — requires holistic review
   e) The school could improve the predictive model by including additional
      variables (work experience, recommendations) to resolve borderline cases.
""")

# ═══════════════════════════════════════════════════════════════════════════
# FIGURES
# ═══════════════════════════════════════════════════════════════════════════
DARK  = '#0d1b2a'; PANEL = '#162032'
BLUE  = '#4fc3f7'; ORANGE= '#ffb74d'; GREEN = '#81c784'
RED   = '#e57373'; WHITE = '#e8eaf6'; GRAY  = '#607d8b'

GRP_COLOR = {1: BLUE, 2: RED, 3: ORANGE}
GRP_LABEL = {1: 'Admitted', 2: 'Not Admitted', 3: 'Borderline'}

def style_ax(ax, title):
    ax.set_facecolor(PANEL)
    for sp in ax.spines.values(): sp.set_edgecolor(GRAY)
    ax.tick_params(colors=WHITE, labelsize=8)
    ax.xaxis.label.set_color(WHITE)
    ax.yaxis.label.set_color(WHITE)
    ax.set_title(title, color=WHITE, fontsize=10, fontweight='bold', pad=6)

def confidence_ellipse(x, y, ax, n_std=1.5, **kwargs):
    cov = np.cov(x, y)
    pearson = cov[0,1] / np.sqrt(cov[0,0]*cov[1,1])
    rx, ry = np.sqrt(1+pearson), np.sqrt(1-pearson)
    ellipse = Ellipse((0,0), width=rx*2, height=ry*2, **kwargs)
    sx, sy = np.sqrt(cov[0,0])*n_std, np.sqrt(cov[1,1])*n_std
    t = transforms.Affine2D().rotate_deg(45).scale(sx, sy).translate(x.mean(), y.mean())
    ellipse.set_transform(t + ax.transData)
    return ax.add_patch(ellipse)

fig = plt.figure(figsize=(18, 16))
fig.patch.set_facecolor(DARK)
gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.48, wspace=0.38)

# ── 1. GPA vs GMAT scatter with ellipses ──────────────────────────────────
ax1 = fig.add_subplot(gs[0, :2])
style_ax(ax1, 'GPA vs GMAT by Admission Status')
for s in [1, 2, 3]:
    sub = df[df.Status == s]
    ax1.scatter(sub.GPA, sub.GMAT, c=GRP_COLOR[s], label=GRP_LABEL[s],
                alpha=0.65, s=50, edgecolors='none', zorder=3)
    confidence_ellipse(sub.GPA.values, sub.GMAT.values, ax1, n_std=1.5,
                       edgecolor=GRP_COLOR[s], facecolor='none',
                       linewidth=2, linestyle='--', zorder=2)
    mx, my = sub.GPA.mean(), sub.GMAT.mean()
    ax1.scatter(mx, my, c=GRP_COLOR[s], marker='*', s=250,
                edgecolors=WHITE, linewidths=0.8, zorder=5)

ax1.set_xlabel('GPA', color=WHITE)
```

```
ax1.set_ylabel('GMAT Score', color=WHITE)
ax1.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax1.axvline(3.30, color=WHITE, linestyle=':', linewidth=1, alpha=0.4, label='GPA=3.30')
ax1.axhline(500, color=WHITE, linestyle=':', linewidth=1, alpha=0.4)

# ■■ 2. Group means bar chart ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax2 = fig.add_subplot(gs[0, 2])
style_ax(ax2, 'Group Means')
x = np.arange(2)
w = 0.25
for i, s in enumerate([1, 2, 3]):
    sub = df[df.Status == s]
    vals = [sub.GPA.mean(), sub.GMAT.mean()/100]  # scale GMAT for visibility
    ax2.bar(x + (i-1)*w, vals, w, color=GRP_COLOR[s], alpha=0.85,
            label=GRP_LABEL[s])
ax2.set_xticks(x)
ax2.set_xticklabels(['GPA\n(actual)', 'GMAT\n(÷100)'], color=WHITE, fontsize=9)
ax2.set_ylabel('Mean Value', color=WHITE)
ax2.legend(facecolor=DARK, labelcolor=WHITE, fontsize=7.5)

# ■■ 3. Discriminant score scatter (F1 vs F2) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax3 = fig.add_subplot(gs[1, :2])
style_ax(ax3, 'Discriminant Score Plot: Function 1 vs Function 2')
for s in [1, 2, 3]:
    mask = y == s
    ax3.scatter(scores[mask, 0], scores[mask, 1],
                c=GRP_COLOR[s], label=GRP_LABEL[s],
                alpha=0.60, s=45, edgecolors='none')
for s in [1, 2, 3]:
    c1, c2 = centroids[s]
    ax3.scatter(c1, c2, c=GRP_COLOR[s], marker='*', s=280,
                edgecolors=WHITE, linewidths=0.8, zorder=5)
    ax3.annotate(f'  {GRP_LABEL[s]}\n  ({c1:.2f}, {c2:.2f})',
                 (c1, c2), color=WHITE, fontsize=7.5, fontweight='bold')
ax3.set_xlabel(f'Function 1 ({explained[0]*100:.1f}% variance)', color=WHITE)
ax3.set_ylabel(f'Function 2 ({explained[1]*100:.1f}% variance)', color=WHITE)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax3.axhline(0, color=GRAY, linewidth=0.6, alpha=0.4)
ax3.axvline(0, color=GRAY, linewidth=0.6, alpha=0.4)

# ■■ 4. Posterior probabilities for each applicant ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax4 = fig.add_subplot(gs[1, 2])
style_ax(ax4, 'Std. Discriminant Coefficients')
variables = ['GPA', 'GMAT']
x4 = np.arange(len(variables))
w4 = 0.35
ax4.bar(x4 - w4/2, std_scalings[:, 0], w4, color=BLUE, alpha=0.85, label='Function 1')
ax4.bar(x4 + w4/2, std_scalings[:, 1], w4, color=ORANGE, alpha=0.85, label='Function 2')
ax4.set_xticks(x4)
ax4.set_xticklabels(variables, color=WHITE, fontsize=10)
ax4.axhline(0, color=WHITE, linewidth=0.8)
ax4.set_ylabel('Standardized Coefficient', color=WHITE)
ax4.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
for i, (c1, c2) in enumerate(zip(std_scalings[:, 0], std_scalings[:, 1])):
    ax4.text(i - w4/2, c1 + (0.02 if c1 >= 0 else -0.05), f'{c1:.3f}',
             ha='center', color=WHITE, fontsize=8)
    ax4.text(i + w4/2, c2 + (0.02 if c2 >= 0 else -0.05), f'{c2:.3f}',
             ha='center', color=WHITE, fontsize=8)

# ■■ 5. Function 1 score distributions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Distribution of Discriminant Function 1 Scores by Group')
for s in [1, 2, 3]:
    ax5.hist(scores[y==s, 0], bins=15, alpha=0.55, color=GRP_COLOR[s],
             label=f'{GRP_LABEL[s]} (centroid={centroids[s][0]:.2f})')
for s in [1, 2, 3]:
    ax5.axvline(centroids[s][0], color=GRP_COLOR[s],
```

```
                linestyle='--', linewidth=2)
ax5.set_xlabel('Discriminant Function 1 Score', color=WHITE)
ax5.set_ylabel('Frequency', color=WHITE)
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

# ■■ 6. Classification accuracy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Classification Accuracy by Group')
group_acc = [cm[i,i]/(y==s).sum()*100 for i,s in enumerate([1,2,3])]
bars = ax6.bar(list(GRP_LABEL.values()), group_acc,
               color=[BLUE, RED, ORANGE], alpha=0.85, width=0.5)
ax6.set_ylim(0, 115)
ax6.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars, group_acc):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
             f'{acc:.1f}%', ha='center', va='bottom', color=WHITE,
             fontsize=10, fontweight='bold')
ax6.axhline(accuracy, color=GREEN, linestyle='--', linewidth=1.5,
            label=f'Overall ({accuracy:.1f}%)')
ax6.axhline(chance, color=GRAY, linestyle=':', linewidth=1.5,
            label=f'Chance ({chance:.1f}%)')
ax6.tick_params(axis='x', labelrotation=10)
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    f"Question 9.9 — Discriminant Analysis of Graduate School Admissions (ADMIS.DAT)\n"
    f"Wilks' Λ={wl:.4f}  χ²={chi2_stat:.2f} (df={df_chi}, p<0.0001)"
    f"  Classification Accuracy={accuracy:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)
plt.savefig('q9_9_admissions_discriminant.png', dpi=150, bbox_inches='tight',
            facecolor=DARK)
plt.close()
print("Figure saved: q9_9_admissions_discriminant.png")
```